

CCNA 200-301 V2.0

28 Troubleshooting Client Connectivity

Part V — Wireless, Virtualisation and Operations

EXAM TOPICS

1.6

LECTURER

Dr. Mustafa Hameed

THE ISLAMIA UNIVERSITY OF BAHAWALPUR

Department of Information Technology

What you will be able to do

- **Read** a host's IP configuration on Windows, macOS and Linux, and recognise the four configurations that cannot work.
- **Apply** a reachability ladder that localises a fault in five commands.
- **Check** wireless security parameters on all three operating systems, including the modern default that breaks MAC-based authorisation.
- **Decide**, from the host alone, whether a fault is the client's, the access layer's, or the network's — before escalating it.

Before we start

Chapter 4 and Chapter 5 for addressing. Chapter 20 for what an address means when it arrives. Chapter 21 for name resolution. Chapter 27 for RSSI, SNR and the security protocols this chapter checks the settings of.

Vocabulary for this chapter

APIPA

default gateway

subnet mask

reachability ladder

ipconfig

ip addr

ARP cache

MAC randomisation

supplicant

EAP method

Each is defined where it first matters — not here.

Three areas, three operating systems, named explicitly

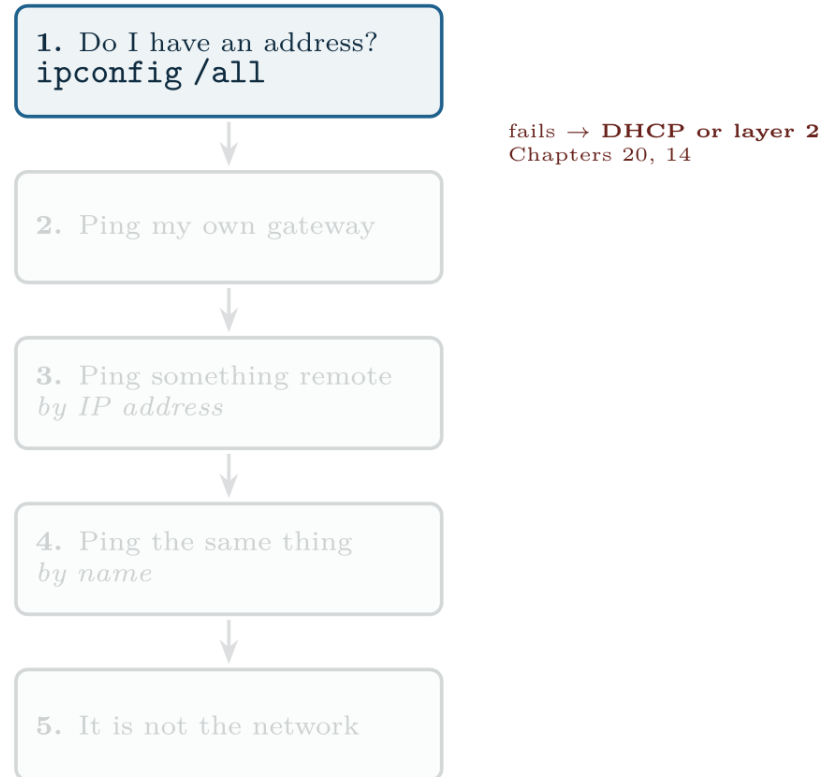
This is what the blueprint actually asks for

Topic **1.6** is *troubleshoot wired and wireless client connectivity (IP configuration, network reachability, and wireless security parameters on Windows, MacOS, and Linux)*.

The three operating systems are named in the topic statement. That is a strong signal: you are expected to know the *equivalent command on each*, not just the one you happen to use. The table in Section sec:oscmds is the single most examinable page in this chapter, and it is worth memorising in all three columns.

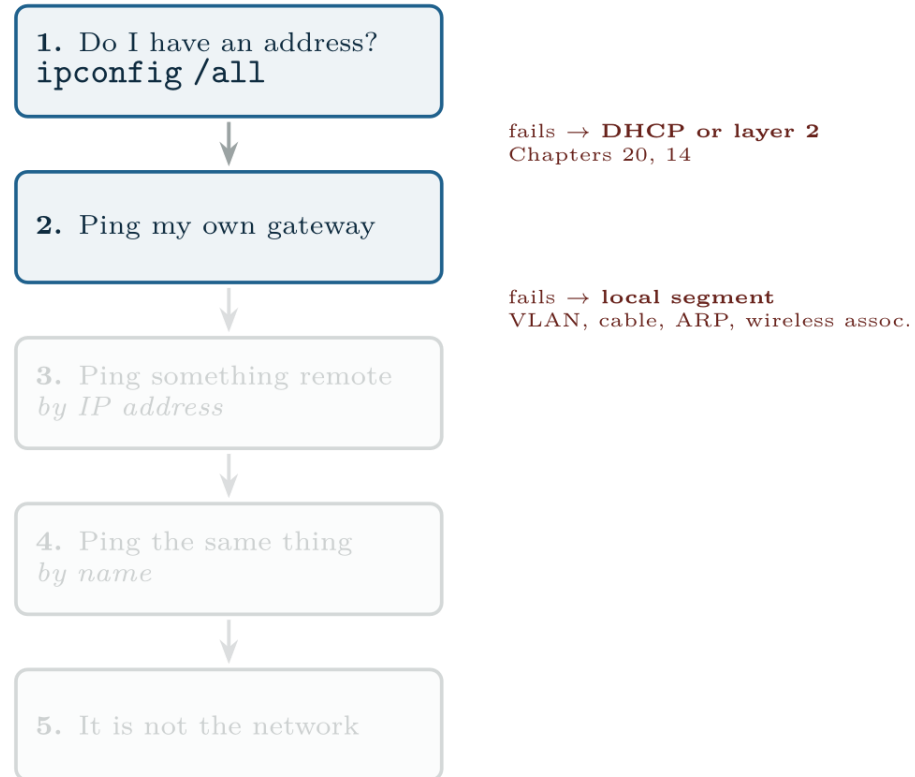
Note also what is *not* here: no server-side configuration, no controller. Everything in this topic is done standing at the affected machine.

Start at the host



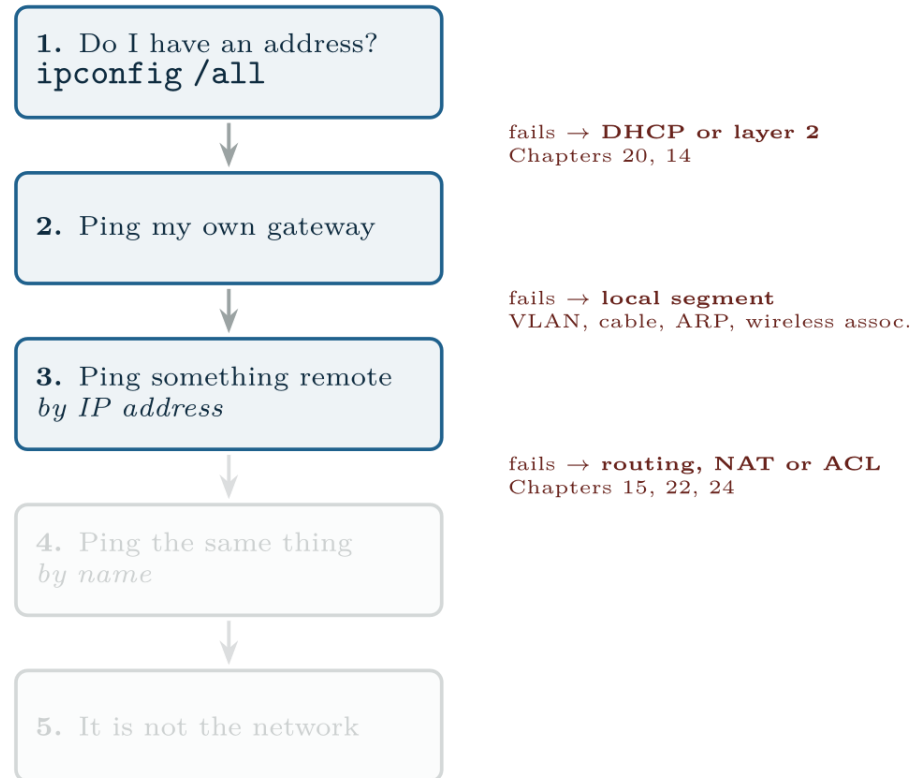
Start at the host, not at the core. No address, and nothing else in this list can possibly work.

Start at the host



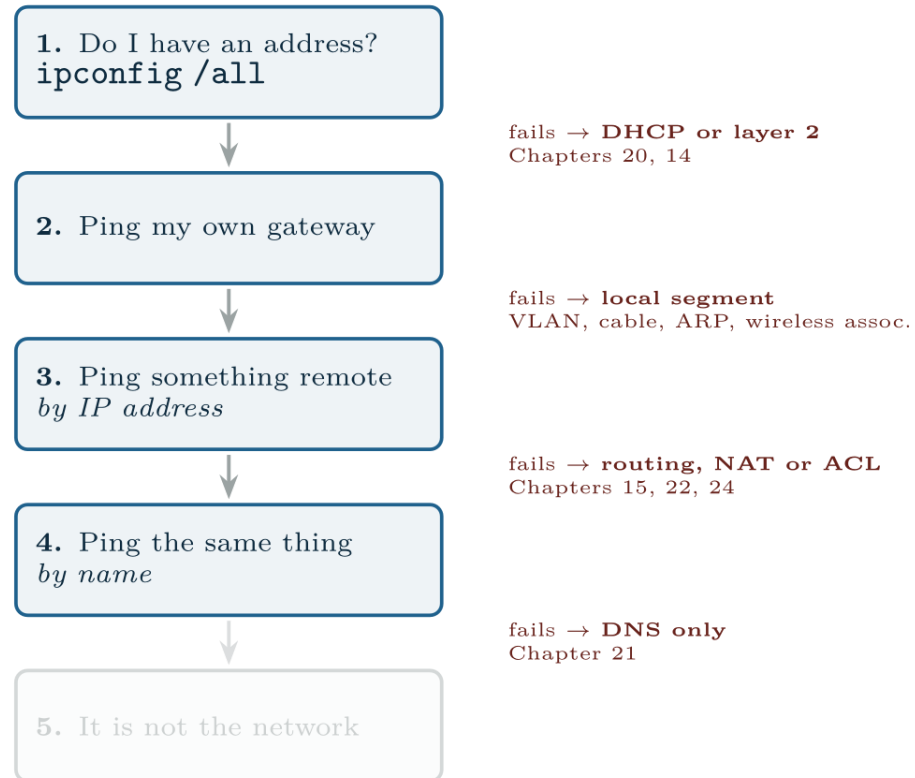
The gateway is the last hop still on our own segment. Passing here clears the cable, the VLAN and the association.

Start at the host



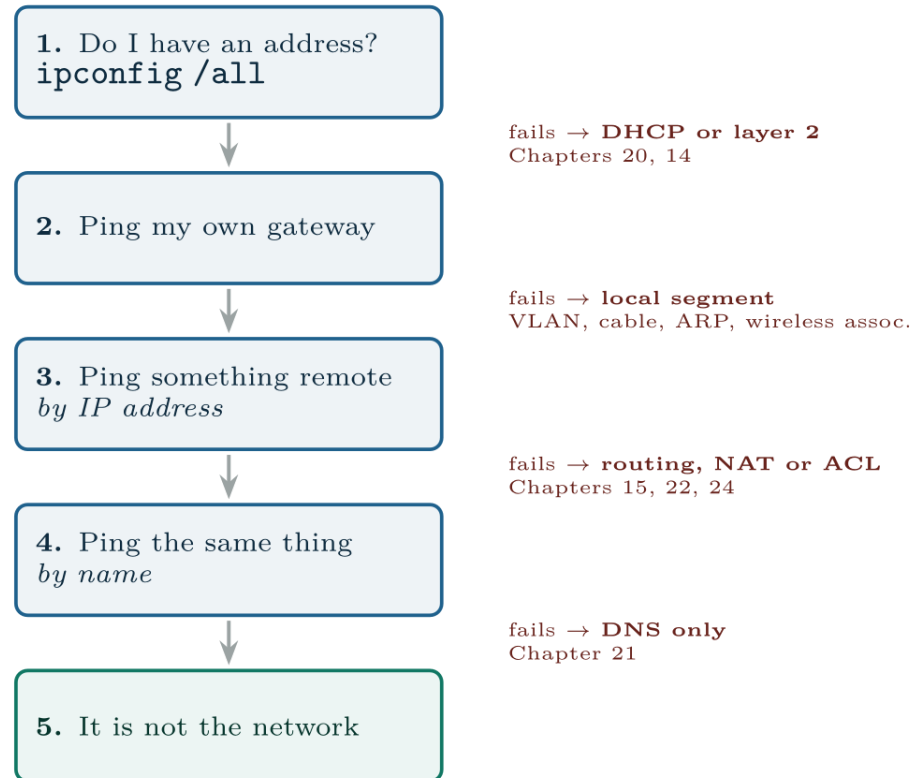
By address, on purpose — it takes name resolution out of the question entirely.

Start at the host



Only now bring names back. If 3 passed and 4 fails, exactly one thing is left, and it is not the network.

Start at the host



Every rung that passes eliminates everything below it. Four commands, and the fault is in one of five places.

Why the order matters

Each step depends on every step above it. So the *first* failure is the fault, and everything below it is a symptom rather than a cause.

The commonest mistake in practice is starting at step 4 — “the website does not load” — and reasoning about DNS for twenty minutes on a machine that never had a valid address. Four commands, in order, cost ninety seconds and stop that happening.

Two differences worth knowing rather than guessing

ifconfig is deprecated on Linux and absent from many modern distributions. `ip` replaced it. On macOS `ifconfig` is still the normal tool. Same command name, opposite advice, depending on the platform in front of you.

macOS does not release and renew the way Windows does. Setting the interface to `NONE` and back to `DHCP` is the equivalent; there is no single “renew” verb. In the GUI it is the Renew DHCP Lease button in the TCP/IP pane.

ipconfig /all

Windows

Wireless LAN adapter Wi-Fi:

```

Description . . . . . : Intel(R) Wi-Fi 6 AX201
Physical Address. . . . . : 8A-3C-D1-77-04-B2
DHCP Enabled. . . . . : Yes
IPv4 Address. . . . . : 192.168.10.34(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Lease Obtained. . . . . : 27 August 2026 08:14:02
Lease Expires . . . . . : 27 August 2026 16:14:02
Default Gateway . . . . . : 192.168.10.1
DHCP Server . . . . . : 10.0.0.5
DNS Servers . . . . . : 192.168.20.10
                        192.168.20.11
    
```

ip addr show wlan0

Linux

```
3: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 state UP  
    link/ether 8a:3c:d1:77:04:b2 brd ff:ff:ff:ff:ff:ff  
    inet 192.168.10.34/24 brd 192.168.10.255 scope global dynamic wlan0  
        valid_lft 21847sec preferred_lft 21847sec  
    inet6 2001:db8:acad:10:8a3c:d1ff:fe77:4b2/64 scope global dynamic
```

The wrong-mask fault, because it is worth recognising

A host configured `192.168.10.34/16` instead of `/24` believes the whole of `192.168.0.0/16` is on its local segment. So for a destination in `192.168.20.0/24` it ARPs directly instead of sending to the gateway — gets no answer, and fails. For `8.8.8.8` it uses the gateway correctly, and works.

The symptom is therefore “the internet works but the file server does not”, which sounds like a server problem and is not. Check the mask whenever connectivity is selectively broken toward *internal* destinations.

ping 192.168.10.1

Windows

```
Pinging 192.168.10.1 with 32 bytes of data:  
Reply from 192.168.10.1: bytes=32 time=1ms TTL=255  
Reply from 192.168.10.1: bytes=32 time=1ms TTL=255  
  
Ping statistics for 192.168.10.1:  
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
```


ip neigh

Linux

```
192.168.10.1 dev wlan0 lladdr 00:1a:2b:3c:4d:5e REACHABLE
192.168.10.22 dev wlan0 lladdr 00:50:79:66:68:22 STALE
192.168.10.99 dev wlan0 FAILED
```

A ping that fails does not always mean unreachable

Plenty of hosts and firewalls drop ICMP by policy. Before concluding that a server is down, test the port the application actually uses:

- Windows: `Test-NetConnection server.example.edu -Port 443`
- macOS and Linux: `nc -zv server.example.edu 443`

A successful TCP connection to port 443 on a host that ignores ping is a working server. Reporting it as down because it did not answer ICMP is a common and avoidable error.

netsh wlan show interfaces

Windows

```
Name           : Wi-Fi
State          : connected
SSID           : CAMPUS-STAFF
BSSID          : 00:11:22:aa:bb:03
Network type   : Infrastructure
Radio type     : 802.11ax
Authentication : WPA2-Enterprise
Cipher         : CCMP
Channel        : 44
Signal         : 78%
```

iw dev wlan0 link

Linux

```
Connected to 00:11:22:aa:bb:03 (on wlan0)
    SSID: CAMPUS-STAFF
    freq: 5220
    signal: -58 dBm
    rx bitrate: 780.0 MBit/s
    tx bitrate: 585.0 MBit/s
```

Windows reports signal as a percentage, and it is not one

That **Signal : 78%** is a rescaling of RSSI, not a physical quantity, and different drivers scale it differently. It also tells you nothing about the noise floor, which Chapter 27 established is the number that decides whether a link works.

Prefer a tool that reports dBm. On Linux **iw** gives it directly; on macOS the option-click menu shows RSSI and noise together; on Windows a third-party analyser is usually needed.

MAC randomisation, and why it breaks things that used to work

Windows, macOS, iOS, Android and most Linux distributions now generate a **random MAC address per network by default**, and some re-randomise periodically. It is a privacy feature and it is on unless somebody turned it off.

It breaks anything that identifies a device by MAC address:

The diagnostic tell: the MAC in `ipconfig /all` does not match the MAC on the device's label or in the network's records, and the second octet's low-order bits mark it as locally administered.

Where to turn it off — and note the topic names all three:

- MAC-based authorisation or filtering — the device is unknown.
- DHCP reservations — the printer that always had `.50` now does not.
- Per-device VLAN assignment.
- Any audit trail that assumed a MAC identified a machine.
- **Windows** — Settings, Network, Wi-Fi, the network's properties: "Random hardware addresses".

MAC randomisation, and why it breaks things that used to work (continued)

- **macOS and iOS** — the network's info panel: "Private Wi-Fi Address".
- **Linux (NetworkManager)** — `wifi.cloned-mac-address` set to `permanent` on the connection profile.

The better long-term answer is to stop authenticating by MAC address at all and use 802.1X, which identifies the *user*. But the exam and the helpdesk both need the short answer.

Common pitfalls

- **Starting at DNS.** Run the ladder in order.
- **Concluding “down” from a failed ping.** Test the actual port.
- **Trusting a signal percentage.** Get dBm, and get the noise floor.
- **Ignoring the subnet mask** when connectivity is selectively broken toward internal hosts.
- **Forgetting that `ifconfig` is deprecated on Linux** and normal on macOS.
- **Assuming a MAC address identifies a device.** It has not been safe to assume that since roughly 2020.

Common pitfalls (continued)

- **Not checking whether it is one host or many.** One host is the host; a whole VLAN is the network. Ask before you start.
- **Fixing it without recording what it was.** A configuration you overwrote is evidence you destroyed.

One laptop associates to the staff wireless, authenticates, and gets no address. Everyone else is fine.

A user's laptop connects to **CAMPUS-STAFF**, shows as connected, and has **169.254.x.x**. The same laptop works on the wired dock. Other laptops in the same room work on the same SSID. The wireless team confirms the client is associated and that 802.1X authentication succeeded.

One laptop associates to the staff wireless, authenticates, and gets no address. Everyone else is fine.

What would you check first — and what do you expect to see?

ipconfig /all

Windows#

Wireless LAN adapter Wi-Fi:

```
Physical Address. . . . . : 8A-3C-D1-77-04-B2
DHCP Enabled. . . . . : Yes
Autoconfiguration IPv4 Address. . : 169.254.88.14(Preferred)
Subnet Mask . . . . . : 255.255.0.0
Default Gateway . . . . . :
```

show ip dhcp snooping binding | include 8a3c

What is the fault — and what does this output rule out?

Why it is doing that

Diagnosis. Work through what has already been ruled out, because it is most of the problem.

Association and 802.1X both succeeded, so the passphrase, the EAP method, the certificate and the RSSI are all fine — the radio link is working and the user is authorised. The wired dock working proves the laptop's TCP/IP stack and the user's credentials are fine. Other laptops working on the same SSID in the same room proves the AP, the WLAN and the DHCP scope are fine.

That leaves something specific to *this device on wireless*. The MAC address is the clue: **8A-3C-D1-77-04-B2**. The first octet, **8A**, is **10001010** in binary — and the second-lowest bit is set, which marks the address as **locally administered** rather than assigned by a manufacturer. This is a randomised MAC.

Why it is doing that

The site assigns the staff VLAN by MAC address for devices in its asset register. The randomised address is not in the register, so the client is placed in a quarantine VLAN with no DHCP scope. Association succeeds, authentication succeeds, and no address is ever offered.

The confirming detail is the empty snooping binding query: the switch has no record of this MAC leasing anything, which is consistent with no offer having been made rather than with an offer being lost.

And the lesson

Immediate fix. Turn off random hardware addresses for this network in Windows Settings, then disconnect and reconnect. The laptop presents its burned-in address, matches the register, lands in the staff VLAN and leases normally.

The real fix is at the other end. MAC-based VLAN assignment stopped being viable when every major operating system began randomising by default. The site already runs 802.1X — authentication succeeded — so it should assign the VLAN from the RADIUS response based on the authenticated *user*, not from a MAC table. That works regardless of what address the client presents, and it is the answer to give when this is reported for the fifth time.

The same fault on three operating systems

Platform note. This lab needs real operating systems rather than a network simulator, because the whole topic is about host-side tools. Virtual machines are fine. If you have only one machine, run Linux and Windows as VMs and use the host as the third.

1. On each of Windows, macOS and Linux, produce the full IP configuration, the routing table and the ARP cache. Build your own version of the command table in Section sec:oscmds from what actually worked — including any command that was missing on your distribution.
2. Run the reachability ladder on each machine while everything works, and record the normal output of each step. You cannot recognise abnormal output without this.
3. **Break 1.** Set a static address with the correct address and a *wrong gateway*. Predict which ladder step fails, then test.
4. **Break 2.** Set the mask to `/16` on a `/24` network. Find two destinations that behave differently and explain the difference using the ARP cache.

The same fault on three operating systems (continued)

1. **Break 3.** Remove the DNS servers. Confirm that steps 1 to 3 pass and step 4 fails, and that this is the *only* configuration that produces that pattern.
2. **Break 4.** Disconnect the network and observe APIPA. Note the address and mask each operating system chooses.
3. Release and renew the lease on all three, using the correct command for each. Note that macOS has no direct equivalent.
4. **Wireless.** Report SSID, BSSID, channel, authentication method and signal on each operating system. Compare what each tool tells you and what it omits.

The same fault on three operating systems (continued)

- 1. MAC randomisation.** Find the setting on each operating system. Record the MAC with it on and off, and confirm the locally-administered bit in the randomised one.
- 2. Extension.** Write the four-step diagnostic script you would give a first-line helpdesk, in the form of exact commands and what each result means. Keep it to one page.

CHECKPOINT 1 of 6

Give the four rungs of the reachability ladder and the area each failure points to.

Discuss · then advance

Checkpoint 1 of 6

Give the four rungs of the reachability ladder and the area each failure points to.

(1) Do I have an address? Failure points to DHCP or layer 2. (2) Ping my gateway — failure points to the local segment. (3) Ping something remote by IP — failure points to routing, NAT or an ACL. (4) Ping the same thing by name — failure points to DNS and nothing else.

CHECKPOINT 2 of 6

Name the command to show the full IP configuration on each of the three operating systems.

Discuss · then advance

Checkpoint 2 of 6

Name the command to show the full IP configuration on each of the three operating systems.

`ipconfig /all` on Windows, `ifconfig` on macOS, `ip addr` on Linux. Note that `ifconfig` is deprecated on Linux and `normal` on macOS.

CHECKPOINT 3 of 6

A host has a correct address and a blank default gateway. What works and what does not?

Discuss · then advance

Checkpoint 3 of 6

A host has a correct address and a blank default gateway. What works and what does not?

It can reach hosts on its own subnet and nothing else. Everything local works, so the user may report it as “the internet is down” rather than as a network fault.

CHECKPOINT 4 of 6

Why does a wrong subnet mask break some internal destinations and not others?

Discuss · then advance

Checkpoint 4 of 6

Why does a wrong subnet mask break some internal destinations and not others?

Because the mask decides which destinations the host believes are local. With a mask that is too short, the host ARPs directly for addresses that are actually on other subnets, gets no answer, and fails — while destinations genuinely outside the over-wide range still use the gateway correctly and work.

CHECKPOINT 5 of 6

A device authenticates to wireless successfully and receives no address. Give two possible causes and how you would tell them apart.

Discuss · then advance

Checkpoint 5 of 6

A device authenticates to wireless successfully and receives no address. Give two possible causes and how you would tell them apart.

Either it is being placed in a VLAN with no DHCP scope — often because MAC randomisation means it is not recognised by a MAC-based authorisation list — or the relay or scope for its VLAN is broken. Tell them apart by checking whether *other* clients on the same SSID and VLAN are leasing normally; if they are, it is specific to this device.

CHECKPOINT 6 of 6

How can you tell a randomised MAC address from a real one by inspection?

Discuss · then advance

Checkpoint 6 of 6

How can you tell a randomised MAC address from a real one by inspection?

The locally-administered bit is set — the second-lowest bit of the first octet. In practice the first octet's second hex digit will be 2, 6, a or e (as in [8A](#)), and the address will not match the device's label or the manufacturer OUI.

What to take away

- Diagnose at the host first. Address, gateway, remote by IP, remote by name — in that order. The first failure is the fault.
- `ipconfig /all` on Windows, `ifconfig` on macOS, `ip addr` on Linux. `ifconfig` is deprecated on Linux and normal on macOS.
- Four broken configurations: no address (`169.254`), no gateway, wrong mask, no DNS. Each has a distinct signature in the ladder.
- A wrong mask breaks internal destinations selectively, which is why it looks like a server fault.

What to take away (continued)

- A failed ping is not proof of a dead host. Test the port with `Test-NetConnection` or `nc -zv`.
- Wireless clients can be associated and authenticated and still have no access. Check the security parameters separately from the radio.
- Windows reports signal as a meaningless percentage. Get dBm and the noise floor.
- MAC randomisation is on by default everywhere and breaks MAC-based authorisation, DHCP reservations and VLAN assignment. Identify it by the locally-administered bit.

Where we got to

- Diagnose at the host first. Address, gateway, remote by IP, remote by name — in that order. The first failure is the fault.
- `ipconfig /all` on Windows, `ifconfig` on macOS, `ip addr` on Linux. `ifconfig` is deprecated on Linux and normal on macOS.
- Four broken configurations: no address (169.254), no gateway, wrong mask, no DNS. Each has a distinct signature in the ladder.
- A wrong mask breaks internal destinations selectively, which is why it looks like a server fault.

NEXT

Chapter 29 — Hypervisors, Virtual Machines and Containers